*Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000. Digital Object Identifier 10.1109/ACCESS.2026.0000000*

# Blockchain-Assisted Zero-Trust Architecture for Secure and Scalable Industrial Internet of Things (IIoT)

[Author Name(s)]
*[Department, Institution, City, Country]*

Corresponding author: [Name] (e-mail: [address]). This work was supported in part by [funding source and grant number].

**ABSTRACT** Industrial IoT (IIoT) networks need per-request verification, but a blockchain on the authentication path adds seconds of latency and saturates ledger capacity. We propose BC-ZTA, a three-tier blockchain-assisted zero-trust architecture in which edge policy enforcement points decide from cached, Merkle-verified ledger state, while identity, trust and audit updates are written asynchronously to a permissioned Hyperledger Besu (IBFT 2.0) network. We implemented the enforcement point and six baselines, including real Keycloak and OpenLDAP services and a central directory with pushed revocations, and measured them on a four-validator network of seven cloud VMs. At 50 devices BC-ZTA answers in 5.4 ms, against 49.1 ms for a centralised directory with a 40 ms network delay and 1382 ms when every request is an on-chain transaction; that synchronous design answers only 40% of requests at 200 devices, whereas BC-ZTA served 200 devices without failures. Event-driven invalidation enforces a revocation in 1.3 s, and all 1750 attack requests were rejected. Batch anchoring cuts audit gas per record 68- to 1351-fold. Keycloak and OpenLDAP revoke in about 84 ms but decide 8× slower at 100 devices. A central directory with pushed revocations and no blockchain matches the latency of BC-ZTA and revokes in 109 ms, so the ledger buys trust distribution and verifiable audit, not speed. Both designs fail closed after twice the cache lifetime when their state source is lost, and a consensus halt delays revocations by minutes. Limitations are a single cloud zone, emulated devices and single-node IAM products.

**INDEX TERMS** Access control, blockchain, Hyperledger Besu, industrial Internet of Things, scalability, smart contracts, zero trust.

## I. INTRODUCTION

### *A. IIoT Security Background*

The Industrial Internet of Things (IIoT) interconnects programmable logic controllers, sensors, actuators, edge gateways and cloud analytics in smart manufacturing, energy and process-control plants. Historically these operational-technology (OT) networks relied on a hardened perimeter and implicit trust between devices on the same segment. As OT converges with IT and remote maintenance becomes routine, that perimeter dissolves, and a single compromised vendor laptop or sensor can become a foothold for lateral movement [1], [2]. Zero Trust Architecture (ZTA), formalised in NIST SP 800-207 [3], replaces location-based trust with per-request verification, and recent systematic reviews confirm its rapid uptake across domains [4]–[6].

### *B. Security Challenges*

Four properties make IIoT hard to secure. (i) *Constrained devices* cannot run heavy cryptography or continuous agents; (ii) *heterogeneity* of protocols and vendors complicates uniform identity management [7], [8]; (iii) *hard real-time budgets* leave only milliseconds for a security decision; and (iv) *scale*, with thousands of devices that join, rotate keys and are revoked continuously. A centralised identity or policy server is both a single point of failure and a high-value target, and conventional logs held by that same operator are not independently trustworthy [5].

### *C. Zero-Trust Requirements*

Following NIST SP 800-207 [3] and critical analyses of ZTA [9], an IIoT deployment needs: **R1** strong, per-device cryptographic identity; **R2** per-request authorisation by a policy decision point (PDP) enforced by a policy enforcement point (PEP); **R3** least-privilege, short-lived sessions; **R4** continuous trust assessment from behavioural and posture signals [10]; **R5** micro-segmentation and containment; and **R6** tamper-evident audit. R1–R5 must be satisfied within control-loop latency budgets.

### *D. Role of Blockchain*

A permissioned blockchain offers a shared, tamper-evident source of truth among mutually distrusting plant, vendor and integrator domains: a decentralised identity registry, policy and reputation state governed by smart contracts, and an immutable audit trail. Prior work has applied it to zero-trust IoT access [11], [12], cross-domain digital-twin access [13], IIoT data protection [14] and contract-based access control [15], [16].

### *E. Research Gap*

Despite this progress, three gaps remain. First, many designs place the ledger *on the request path*; a published Ethereum-based zero-trust framework, for instance, reports latency growing from 49.33 ms to 74.0 ms and throughput falling from 50 to 30.77 req/s [17]. Second, scalability is seldom evaluated for authentication and authorisation workloads, with notable exceptions that rely on sharding [18]; most studies evaluate accuracy or encryption cost [19], [20]. Third, continuous tamper-proof audit logging, repeatedly named as an open gap [5], is expensive when every event is written on-chain.

### *F. Research Objectives*

We aim to (O1) design a blockchain-assisted ZTA whose per-request decisions do not wait for consensus; (O2) bound the resulting staleness of revocation and trust state; (O3) make audit anchoring cost sub-linear in the event rate; and (O4) evaluate latency, on-chain load, gas cost and revocation delay against five baselines, two of them real identity products.

### G. Contributions

- **Architecture.** A three-tier BC-ZTA in which the PEP/PDP decide from edge-cached, Merkle-verified ledger state, so the blockchain is *off* the request path (Section III).
- **Smart contracts.** Four Solidity contracts (identity registry with epoch commitments, quorum-based trust reputation, ABAC access control, audit log with Merkle batch anchoring), compiled and unit-tested with 15 passing tests.
- **End-to-end prototype and measurements.** A running PEP/PDP service, five baselines (a centralised directory, local ABAC, a synchronous blockchain, and real Keycloak and OpenLDAP services, the latter two with and without PEP-side caching) and a load generator, evaluated on a seven-VM Google Cloud deployment with a four-validator Hyperledger Besu IBFT 2.0 network in five repetitions: latency and goodput up to 200 devices, revocation delay, security tests, a validator-failure test, a central-service outage test, a ledger-flood sweep and gas (Sections 5–6).
- **Security analysis and limits.** A property-by-property argument under explicit assumptions, with the residual risks we found (replay across gateways, trust in a single ledger node, audit completeness; Section IV-F), and a measured comparison that shows where the ledger does and does not help (Section VI-H).
- **Bounded-staleness analysis.** Event-driven invalidation reduces mean revocation delay from 6.6 s to 1.3 s (Section VI-F).

The remainder of the paper reviews related work (Section II), presents the architecture (Section III), the use case and threat model (Section IV), the methodology (Section V), results (Section VI) and conclusions (Section VII).

## II. RELATED WORK

### A. Blockchain-Based IIoT Security

Essaid and Ju combine blockchain, edge computing and smart contracts for IIoT data integrity and threat detection [14]. Hyper-IIoT proposes a smart-contract-inspired access-control scheme for resource-constrained devices [15], and formal model checking of IoT access-control contracts has been explored [16]. Lightweight authentication using blockchain and Chebyshev chaotic maps targets cross-domain IIoT [21]. For identity, Asim *et al.* use epoch-based batch registration with sparse Merkle trees to support up to 7,000 devices over 20 fog nodes with under 5 MB on-chain overhead [8], and IOTA-based decentralised identifiers have been prototyped on Raspberry Pi 4 devices, with CPU load peaking during DID generation [22]. DAG ledgers have been proposed for scalability and privacy in intelligent buildings [23], and a smart-city trust framework reports 450–500 tps with a lightweight adaptive PoS [24].

### B. Zero-Trust IIoT Security

Federici *et al.* design a zero-trust remote-access architecture for IIoT [1], and Zanasi *et al.* a flexible architecture combining micro-segmentation, SDN and a central policy layer [2]. Ma and Chiu propose a risk-based access-control engine with continuous neighbour-trust assessment for IoT [10]; Mthethwa *et al.* address non-human identities [7]. Broader treatments include a ZTA framework for next-generation IoT [25], trust-driven adaptive edge resource management that lowers latency by 28.7% against cloud processing [26], and zero-trust foundation models for IoT AI [27]. Surveys summarise applications and challenges [28].

### C. Blockchain-Assisted Zero Trust

Aleisa couples blockchain-enabled ZTA with post-quantum and zero-knowledge techniques, reporting 98% privacy preservation [11]. Nie *et al.* use smart contracts and inner-product encryption for 6G IoT access control, with 14% higher encryption efficiency and 18% lower access latency [12]. Sharma *et al.* add federated transfer learning for Industry 5.0 [19], and Mishra *et al.* a zero-trust digital twin with hash-chained lightweight blockchain auditing and deep-learning intrusion detection [20]. Salim *et al.* reach 2,500 TPS at 700 nodes through DRL-driven sharding [18]. Domain variants exist for FinTech [17], clinical trials with T-PBFT [29] and FPGA supply chains [30].

### D. Non-Blockchain Foundations and Alternatives

Several mechanisms used here, and several competing designs, do not need a blockchain. Revocation on the web is handled by CRLs [31] and OCSP [32], where a server can staple a signed status response to the handshake [33]; a central authority that pushes signed revocations to edge enforcement points is therefore a natural alternative to event-driven invalidation from a ledger. Audit logs that can be checked by third parties are provided by Merkle-tree transparency logs such as Certificate Transparency [34], which rely on log operators and monitors rather than a consensus protocol. Decentralised identifiers (DIDs) [35] define identifiers whose control is separated from any registry operator, and blockchain-based access control for IoT, from FairAccess [36] to scalable management architectures [37], places the policy decision or the policy store on the ledger. For the ledger itself, permissioned platforms offer either execute-order-validate (Hyperledger Fabric [38]) or EVM-based Istanbul BFT [39] on which we build; both derive from classical Byzantine agreement [40]. On the IIoT side, large-scale measurements of OPC UA deployments found many with missing or weak authentication [41], which motivates per-device cryptographic identity (R1). We compare against central directories, real Keycloak and OpenLDAP services and, where noted, a central directory with push revocation, to test whether the ledger improves anything that such designs cannot.

### E. Comparative Analysis

Table 1 compares representative works on four criteria taken from their reported content.

TABLE 1. COMPARISON OF RELATED WORK

| Work | Blockchain | Zero trust | IIoT focus | Scale eval. |
|---|---|---|---|---|
| [18] | Yes | Yes | No | Yes |
| [11] | Yes | Yes | No | No |
| [12] | Yes | Yes | No | Part |
| [19] | Yes | Yes | Part | No |
| [20] | Yes | Yes | Yes | Part |
| [13] | Yes | Part | Yes | Part |
| [15] | Yes | No | Yes | No |
| [8] | Yes | No | Yes | Yes |
| [10] | No | Yes | No | No |
| [2] | No | Yes | Yes | No |
| **This work** | Yes | Yes | Yes | Yes |

Yes = addressed; Part = partly; No = not addressed. Authors' reading of each paper's reported content; "Scale eval." means device/node scaling reported.

Three observations follow. Most blockchain-based zero-trust designs target generic IoT rather than IIoT; only two of the listed works report device or node scaling [8], [18]; and none of the listed works both reports scaling and quantifies the staleness introduced by off-path state. The last point is where this paper concentrates its evaluation.

### F. Research Gap

No work in our (non-systematic) survey simultaneously (a) keeps the ledger off the per-request path, (b) quantifies the revocation-staleness this causes and how to bound it, and (c) amortises audit cost via Merkle anchoring for an IIoT workload. This paper targets that combination.

## III. PROPOSED ARCHITECTURE

### A. Design Objectives

**D1** decisions from local state in milliseconds; **D2** no single party that can change identity records, policy or logs without detection (a plant authority still enrols devices, Section IV-B); **D3** bounded staleness of cached state; **D4** write load that grows slower than the request rate; **D5** a device-side workload of one ECDSA signature per request and one stored Merkle path (device-side cost was not measured, Section VI-L).

### B. Architecture Overview

Fig. 1 shows three tiers. Devices form Layer I (IIoT and edge). Layer II is the zero-trust security layer at the edge gateway, hosting the PEP, PDP and trust engine. Layer III is the permissioned blockchain, whose identity, trust, access-control and audit services are implemented as smart contracts. Gateways cache Layer III state, so Layer II never waits for consensus.

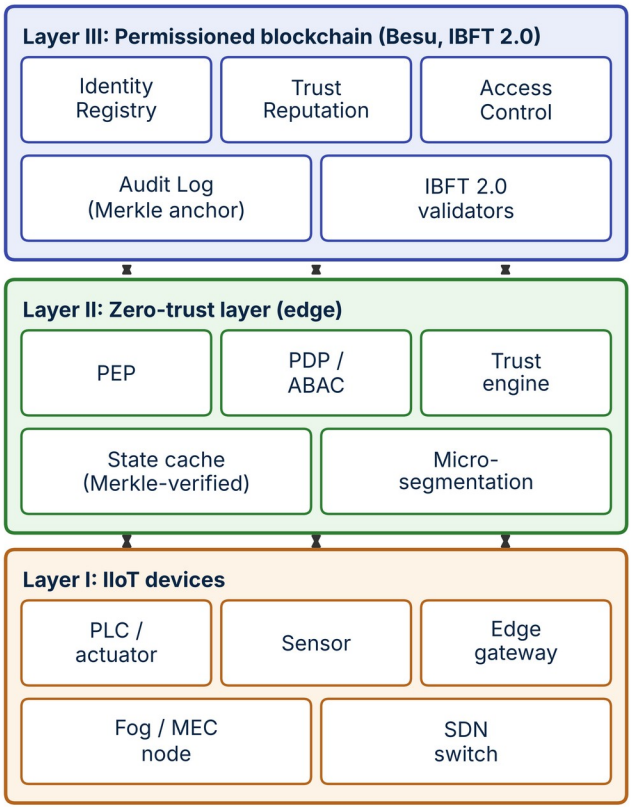



Fig. 1. Proposed BC-ZTA architecture: devices, edge/ZT layer, blockchain layer and contract services.

### C. IIoT/Edge Layer

Sensors, actuators and PLCs hold an ECDSA key pair bound to a DID and sign each request with a timestamp and nonce. Edge gateways terminate sessions, host the PEP, keep a local cache of the identity, trust and policy state, and forward anomaly telemetry to the trust engine.

### D. Zero-Trust Security Layer

The PEP intercepts every request. The PDP verifies the signature, checks the DID against a cached Merkle-verified registry entry, retrieves the trust score and policy, and returns allow/deny. Micro-segmentation is enforced by permitting only flows that hold a valid, unexpired session [2]. A trust engine converts behavioural features (rate, protocol conformance, firmware attestation) into score proposals.

### E. Blockchain Layer

A permissioned Hyperledger Besu network with IBFT 2.0 consensus is operated by plant, integrator and vendor validators; with $n$ validators it tolerates $f = \lfloor (n-1)/3 \rfloor$ Byzantine nodes. Gateways are read-only clients that subscribe to events; only authorised registrars, validators and the policy administrator send transactions.

### F. Identity Management

Each device is a DID with a public-key hash and device class. Registrations are submitted by a plant-authorised registrar and may be grouped into *epochs*: the registrar commits a Merkle root of an epoch batch and devices later prove membership with a logarithmic-size proof, an approach inspired by epoch-based IIoT identity schemes [8]. Key rotation and revocation are single transactions that emit events.

### G. Trust Management

Scores lie in [0,100] with an initial value of 70. A score changes only when a quorum of validators proposes the same value, and updates are rate-limited. An anomaly flag immediately subtracts 20 points (floored at 0) pending re-evaluation, giving fast conservative reaction.

### H. Access Control

Policies are attribute-based: for each resource the administrator defines allowed device-class range, minimum trust score and session TTL. A request is granted iff the DID is active, the class is in range and the trust score meets the threshold; the grant yields a session identifier that expires after the TTL or on explicit revocation.

### I. Smart Contracts

*IdentityRegistry* (register, update key, revoke, epoch commit, membership check); *TrustReputation* (quorum voting, anomaly flag); *AccessControl* (policy, evaluate, session grant/revoke); *AuditLog* (per-event log, Merkle batch anchor, inclusion verification). Table 2 lists the interfaces, and source and tests are in the accompanying package.

TABLE 2. SMART-CONTRACT INTERFACES

| Contract | Main functions | Writer |
|---|---|---|
| IdentityRegistry | registerDID, updateKey, revokeDID, commitEpoch; verifyMembership, isActive (views) | Plant authority |
| TrustReputation | updateScore (quorum), flagAnomaly; getScore (view) | Validators |
| AccessControl | setPolicy, revokeSession; evaluateRequest (view), grantSession, isSessionValid | Admin; any caller |
| AuditLog | logEvent, anchorBatch; verifyInBatch, getAnchor (views) | Open in prototype |

"Open in prototype" means the function has no caller restriction; a deployment would restrict it to gateway accounts.

### J. Continuous Verification

Fig. 2 summarises the workflow. Verification does not end at login: every request is re-evaluated against current cached trust, sessions are short-lived, and the gateway refreshes cached entries every *TTL* seconds or immediately on a *DIDRevoked* or *ScoreUpdated* event. Audit records are hashed into a batch whose Merkle root is anchored on-chain asynchronously (step 7), so logging never delays the response.

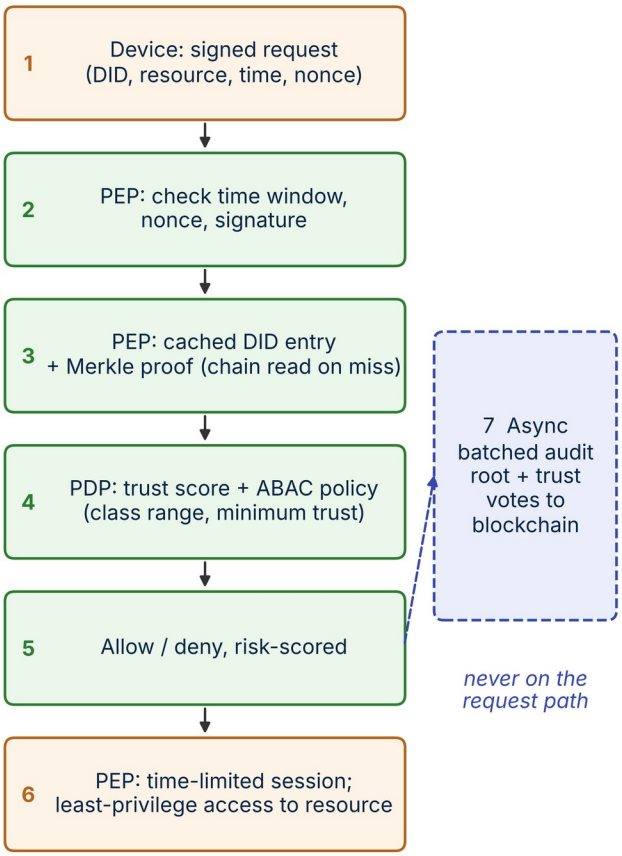



Fig. 2. Per-request verification workflow with cached state reads and asynchronous audit anchoring (step 7).

A request is $r$ = (DID, resource, operation, $t$, $n$, σ), where $t$ is a timestamp, $n$ a nonce and σ the device's ECDSA signature over the other fields. The PEP accepts $r$ only if (i) $|\text{now} - t| \le \Delta$ and $n$ has not been seen within Δ, which bounds replay; (ii) the DID is active in the cached registry, proved by a Merkle path against the latest committed epoch root or by a direct chain read on a cache miss; (iii) σ verifies under the public key whose hash is registered; and (iv) the PDP finds the trust score and policy satisfied. Otherwise the request is denied and an anomaly event is queued. As a fail-closed rule, a gateway that cannot read the ledger keeps using a cached entry after its TTL for at most $k \cdot TTL$ after the entry was last read ($k$ = 2 by default, configuration parameter STALE_FACTOR) and denies every request that needs older state (reason "unavailable") until the ledger is reachable again; $k$ = 1 gives strict fail-closed behaviour. A circuit breaker avoids waiting for the read timeout on every request while the source is down. The first cloud experiments (Sections 6.1–6.9) ran a prototype that denied as soon as a failed read followed TTL expiry ($k$ = 1); the $k$ = 2 rule is the

one described here, and the $k = 2$ rule is evaluated in Section VI-J.

### K. Ledger Access, Revocation and Read Quorum

A Merkle membership proof against an epoch root shows that a DID was enrolled in that epoch; it stays valid after the device is revoked and therefore says nothing about freshness. Revocation is enforced through the registry record (revoked flag), which the gateway reads on a cache miss and which the *DIDRevoked* event invalidates. A gateway also trusts the Besu node it reads from: a compromised node could return stale or forged state. Deployments that do not run a validator at the gateway can read from $f + 1$ nodes pinned to the same block height and accept a state only if all answers agree; any $+ 1$ matching answers include an honest node, and IBFT 2.0 has instant finality, so there are no forks to reconcile. The prototype implements this option (parameter QUORUM = 2 for $n = 4$); the default configuration reads from one node, and the cost of the quorum is reported in Section VI-J.

## IV. USE CASE AND THREAT MODEL

### A. Use Case

A smart factory has several production cells, each with PLCs and sensors behind one edge gateway; a remote vendor engineer needs time-limited maintenance access to one PLC class; a SCADA historian consumes sensor data. Cells, vendor and integrator are separate administrative domains that jointly operate validators.

### B. Assets and Trust Assumptions

Assets are device identities, policy and trust state, session tokens and audit records. We assume a majority-honest validator set (at most $f$ Byzantine), secure device key storage, and gateways that are trusted for enforcement but may be compromised individually.

### C. Adversary Model

A Dolev–Yao network adversary can eavesdrop, replay and inject messages; may compromise a subset of devices (including their keys) and one gateway; and may include an insider with administrative credentials in one domain but not in a validator quorum.

### D. Threats and Countermeasures

Table 3 maps threats to mechanisms.

TABLE 3. THREATS AND COUNTERMEASURES

| Threat | Attack | Countermeasure |
|---|---|---|
| Spoofing | Forged device identity | DID + ECDSA signature; Merkle-verified registry entry |
| Replay | Re-sent request | Timestamp + nonce window at PEP; session TTL |
| Lateral movement | Compromised device probes peers | Per-resource ABAC, micro-segmentation, short sessions |
| Insider / tamper | Admin edits policy or logs | On-chain policy and Merkle-anchored audit; validator quorum |
| Trust poisoning | False score votes | Quorum votes, rate limit; non-validators rejected |
| Stale state | Use of revoked identity | Event-driven cache invalidation, bounded TTL |
| Ledger DoS | Flooding the chain | Off-path reads; batched writes; non-validators cannot write trust |

### E. Security Goals

Authenticity of devices, least-privilege authorisation, revocation within a bounded delay, non-repudiable tamper-evident audit, and availability of access decisions during ledger congestion.

### F. Security Analysis

We argue each goal under the assumptions of Sections 4.2–4.3 plus: ECDSA over secp256k1 is existentially unforgeable under chosen-message attack, Keccak-256 is collision- and second-preimage-resistant, device clocks and gateway clocks differ by less than $\Delta/2$, and the enrolment authority registers only genuine devices. These are proof sketches, not machine-checked proofs; a ProVerif [42] model of the request protocol is left for future work.

**S1 Authenticity.** A request is accepted only if the signature recovers an address whose hash equals the key hash registered for the claimed DID. A forger without the device key must break unforgeability; registering a new key needs the authority (*updateKey*/*registerDID*). **S2 Replay.** A replayed request is rejected if it is older than $\Delta$ or its nonce was seen by *the same gateway* within $\Delta$. The signed message does not contain a gateway identifier, so the same request can be replayed once at a different gateway within $\Delta$ unless nonce caches are shared or the gateway identity is added to the signed fields; we did not test or implement this mitigation. **S3 Revocation.** After a revocation transaction is finalised (IBFT 2.0 has instant finality), a gateway that observes the event stops accepting the device within the event-polling interval; a gateway that misses the event enforces it when its cache entry expires, i.e. within *TTL*. If consensus is halted the transaction cannot be finalised, so the bound is measured from finalisation, not from submission (Section VI-J). **S4 Audit integrity.** Each batch root is anchored on-chain; a stored record that is altered later no longer verifies against its batch root, and an anchored root cannot be changed without $f + 1$ validators. Audit completeness is not guaranteed: a compromised gateway can omit records before they are anchored (up to the anchoring interval of 1 s is also exposed on a crash), and the prototype's *AuditLog* accepts anchors from any account. **S5 Availability.** Decisions never wait for consensus and, when the ledger is unreachable, continue from cache for at most $k \cdot TTL$ (Section III-J); a gateway whose node is compromised can deny service or serve stale state unless the read quorum of Section III-K is used. **Non-goals.** Compromise of the enrolment authority, of $f + 1$ validators, of a device key before revocation, and traffic analysis.

## V. IMPLEMENTATION AND EXPERIMENTAL METHODOLOGY

### A. Besu Blockchain Network

We deployed a four-validator Hyperledger Besu v26.9.0 IBFT 2.0 network ($f = 1$, block period 2 s, chain ID 2026, zero base fee, London EVM) with Docker on a GitHub-hosted 2-vCPU Linux runner for the contract-level micro-benchmark described below; all validators share that single host, so network delay is negligible and CPU is contended. The end-to-end experiments of Section V-C were run separately on a seven-VM Google Cloud deployment (Section V-G). A fresh network was started for each of 3 micro-benchmark repetitions. Contracts were compiled with Solidity 0.8.20

(optimizer 200 runs, EVM "london"). Five scenarios were run with pre-signed transactions from 8 worker wallets: S1 sequential write confirmation, S2 read latency (500 *eth_call* evaluations of *AccessControl.evaluateRequest*), S3 per-event logging at 10–200 tx/s offered for 20 s, S4 batch anchoring at 1 tx/s, and S5 revocation visibility. Block observation was polled every 25 ms. The scripts, contracts and raw results are in the public code repository listed under Data and Code Availability.

### *B. Smart Contracts*

The four contracts total about 370 lines. A Hardhat/Chai suite of 15 tests passes and covers registration, duplicate and unauthorised registration, revocation, key update, quorum scoring, score floor, rate limiting, non-validator rejection, access grant/deny, session expiry, epoch Merkle membership and audit batch inclusion proofs.

### *C. End-to-End Prototype and Load Generator*

Besides the contract-level tests, we implemented the complete request path as running software (Node.js 20): a PEP/PDP service, a load generator and the baselines of Section V-D. Each device owns a secp256k1 key and signs *DID* | *resource* | *timestamp* | *nonce*; its key hash and device class are registered on-chain and its DID is part of one Merkle epoch commitment. The PEP rejects stale timestamps (>5 s) and repeated nonces, recovers the signer, checks it against the registered key hash, verifies the Merkle membership proof against the cached epoch root, and evaluates the ABAC policy (device class 0–1, trust score ≥ 50). In BC-ZTA, identity, trust and policy state are read from Besu with *eth_call* on a cache miss and cached for 30 s; a block-polling subscription (20 ms) invalidates an entry when a *DIDRevoked*, *KeyUpdated* or *ScoreUpdated* event is mined; every decision is hashed, and once per second the Merkle root of the pending batch is anchored with *AuditLog.anchorBatch*. The load generator emulates N = 10, 25, 50, 100, 200 devices, each sending one request per second with staggered phases, for 5 s of warm-up and a 20 s measurement window; requests are signed before the window so that the generator itself is not the bottleneck. In the end-to-end experiments the PEP, the directory server, the load generator and the four Besu validators run on separate machines (Section V-G).

### *D. Baselines*

(B1) *Centralised PKI-IAM*: each request makes one lookup at a separate directory server, on a separate directory VM whose responses are delayed by 40 ms with Linux *tc netem* to model a remote central service (about 50 ms per lookup in total, Section VI-A). (B2) *ABAC without blockchain*: a local attribute store at the PEP. (B3) *Blockchain-only (sync)*: for every request the PEP reads the state with *eth_call* and sends an *AuditLog.logEvent* transaction (eight sender accounts) and replies only after the block containing it is observed. (B4) *Keycloak*: a real Keycloak 26.4.0 identity provider issues each device an OpenID Connect access token carrying its class, trust score and key hash as claims, and the PEP calls the token-introspection endpoint [43] on every request; revocation disables the user. (B5) *OpenLDAP*: a real slapd directory holding the same attributes, queried once per request [44]. B4 and B5 are also run with a PEP-side decision cache ("cached", TTL 10 s), which is the usual way to cut their latency. (P) *Proposed BC-ZTA* as in Section V-C. All systems use the same request format, signature check and policy, so differences come from where state is read and written.

### *E. Experimental Parameters*

Table 4 lists the experimental parameters; the injected network delays are assumptions, the other values are configuration of the prototype.

TABLE 4. EXPERIMENTAL PARAMETERS

| Parameter | Value |
|---|---|
| Ledger | Besu v26.9.0, IBFT 2.0, 4 validators |
| Block period | 2 s |
| Platform | Google Cloud, 7 VMs (Section V-G) |
| Devices; request rate | 10–200; 1 req/s each |
| Warm-up; window; runs | 5 s; 20 s; 5 |
| Cache TTL; anchor interval | 30 s; 1 s |
| Directory delay (B1, netem) | 40 ms |
| Validator link delay (netem) | 5 ms |
| Freshness window; policy | 5 s; class 0–1, trust ≥ 50 |
| Revocation tests | 8 devices × 5 runs per system; TTL 10 s |

Network delay is injected with tc netem on the directory and validator VMs only; everything else is measured on running software.

### *F. Performance Metrics*

(E1) End-to-end latency (mean, P95) from request send to decision, goodput (accepted requests per second), failures, and on-chain writes per second versus device count. (E2) Revocation enforcement delay: time from the revocation action until the PEP first denies the device. (E3) Security tests with 50 attempts each per run. (E4) Decision latency while the ledger is flooded with transactions. (E5) Decision latency, goodput and revocation enforcement while one of the four validators is stopped. Also gas per operation and, from the micro-benchmark, confirmation latency and ledger throughput.

### *G. Distributed Deployment*

The end-to-end experiments E1–E5 ran on Google Cloud (zone us-central1-a) on seven separate virtual machines in one VPC without external addresses: four Besu validators (e2-medium, 2 vCPU, 4 GB; one validator each), one PEP/PDP host (e2-standard-2), one directory server for B1 (e2-small) and one controller (e2-standard-2) that runs the load generator and the experiment scripts and submits the on-chain transactions. Linux *tc netem* delays all outgoing traffic of each validator VM by 5 ms to represent a plant-local validator network, and the responses of the directory server by 40 ms to represent a remote central service; all other delay is the real VM-to-VM network delay. Each of the 5 repetitions started a freshly generated chain and restarted the PEP for every device count; the repetitions ran sequentially on the same VMs. The validator-failure test (E5) stops one validator container for 30 s while 50 devices are served. Cloud VMs in one zone approximate independent hosts better than a shared CI runner, but they are not an industrial network: devices are emulated by the load generator and co-tenancy in the cloud can add variance. The deployment

scripts, configuration and raw results are in the public code repository listed under Data and Code Availability.

A second cloud run on the same seven VMs added Keycloak 26.4.0 (Docker, development mode, embedded H2 database, OIDC token introspection) and OpenLDAP slapd (Debian 12 package, one search per request) on the directory VM (B4 and B5), the PEP-cache variants of both, a revocation test, a 25 s outage test, an open-loop PEP capacity test and the ledger flood sweep of Section VI-I. Besu was started with a 2.5 GB heap for the flood sweep and for all later runs; the first IAM repetition used the default 1 GB heap, which does not affect the PEP-side measurements reported here. The scripts, orchestration and raw results are in the code package.

## VI. RESULTS AND DISCUSSION

### A. Authentication

Table 5 and Fig. 3 show end-to-end latency (mean of 5 runs). At 50 devices BC-ZTA answers in 5.4 ms on average (P95 6.2 ms), versus 49.1 ms for the centralised PKI-IAM, 5.2 ms for the ledger-free ABAC store and 1382 ms for the synchronous blockchain. BC-ZTA is therefore 89% faster than the centralised baseline, costs about 0.2 ms more than a local store (Merkle verification, hashing and audit bookkeeping), and is about 255× faster than checking every request on-chain. Even at 10 devices, where the ledger is not saturated, the synchronous design needs 1172 ms because each request waits for a block.

TABLE 5. END-TO-END LATENCY (MS, MEAN ± 95% CI OF 5 RUNS; P95)

| System | Mean 50 | P95 50 | Mean 200 | P95 200 |
|---|---|---|---|---|
| PKI-IAM | 49.1 ± 0.2 | 50.0 | 49.0 ± 0.2 | 50.9 |
| ABAC | 5.2 ± 0.3 | 6.0 | 4.1 ± 0.2 | 5.1 |
| BC-only | 1382 ± 17 | 2272 | 12094 ± 6401 | 20757 |
| BC-ZTA | **5.4 ± 0.1** | **6.2** | **4.6 ± 0.2** | **5.8** |

Columns give the number of devices; PKI-IAM responses are delayed by 40 ms (netem). With 5 runs the differences discussed in the text exceed the confidence intervals by an order of magnitude; no hypothesis tests were run.

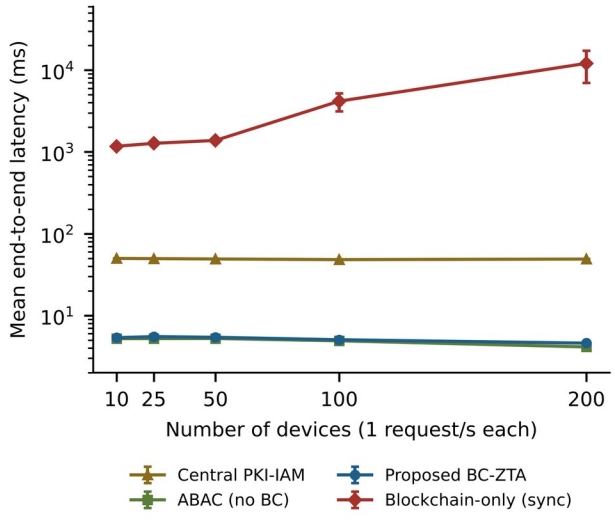



Fig. 3. Mean end-to-end latency versus number of devices (log scale; error bars: standard deviation of 5 runs).

### B. Authorization

The proposed design makes policy decisions from the edge cache. The measured cache-hit ratio was 96% (one miss per device per 30 s TTL), so 50 devices caused only 50 chain reads for about 1,250 requests; each miss costs one batched *eth_call* of 5.4 ms in the micro-benchmark. The PEP used 0.29 CPU cores at 50 devices and 0.65 at 200, against 0.53 for the local store. Every decision is covered by the audit batch: in the 200-device runs about 5,000 decisions per run were anchored in roughly 25 transactions. On-chain evaluation costs 44,685 gas as an estimate, but because it is a read, no fee or block wait is incurred.

### C. Blockchain Throughput

Table 6 summarises the Besu micro-benchmark. Write confirmation takes 1522 ms on average at 10 tx/s (P95 2439 ms), close to the 2 s block period; a transaction submitted just after a block, as in the phase-locked sequential tests S1/S5, waits about 1980 ms. Reads cost 5.4 ms (P95 10.9 ms). Committed throughput tracks the offered load up to 50 tx/s (46.5 tx/s) but saturates near 60 tx/s on this host: at 100 and 200 tx/s offered only 62.7 and 59.1 tx/s commit and 43% of transactions at 200 tx/s were not confirmed within 60 s. Batch anchoring at 1 tx/s confirmed all 60 anchors in 941 ms on average. Because all validators share one 2-vCPU host, 60 tx/s is a lower bound: on the seven-VM deployment the synchronous baseline committed up to 112 tx/s (Section VI-E), almost twice as much.

TABLE 6. MEASURED BESU RESULTS (SHARED CI HOST, 3 RUNS, MEAN)

| Offered (tx/s) | Commit (tx/s) | Mean (ms) | P95 (ms) | Conf. |
|---|---|---|---|---|
| 10 | 9.5 | 1522 | 2439 | 100% |
| 25 | 23.3 | 1706 | 2659 | 100% |
| 50 | 46.5 | 2330 | 3626 | 100% |
| 100 | 62.7 | 11601 | 19753 | 100% |
| 200 | 59.1 | 16424 | 27044 | 57% |

Per-event logging (S3), four IBFT 2.0 validators on one shared 2-vCPU host; Conf. = confirmed fraction.

### *D. Resource Utilization*

Measured gas per operation is listed in Table 7. Registering a device costs 55,682 gas, whereas committing an epoch root costs 75,469 gas independent of batch size. Logging one event on-chain costs 182,008 gas, while anchoring a batch costs 134,747 gas, i.e. about 135 gas per record for a 1,000-record batch (≈1351× cheaper); the saving is proportional to the batch size and is ≈68× for the 50 records per batch of the 50-device experiments and ≈270× for 200 devices. At 400 devices and 1 event/s the calculation gives 34.6 M events/day: 6636 MB/day of ledger growth with per-event logging (192 B/entry) versus 13.8 MB/day with batch anchoring. Device-side CPU and memory were not measured; prior IOTA-based DID work reports that DID generation can saturate a Raspberry Pi 4 CPU [22], which motivates our use of hash proofs only.

TABLE 7. MEASURED GAS PER CONTRACT OPERATION

| Operation | Gas |
|---|---|
| IdentityRegistry.registerDID | 55,682 |
| IdentityRegistry.commitEpoch | 75,469 |
| IdentityRegistry.revokeDID | 49,550 |
| AccessControl.setPolicy | 49,731 |
| AccessControl.grantSession | 71,689 |
| TrustReputation.updateScore (quorum) | 124,772 |
| TrustReputation.flagAnomaly | 95,868 |
| AuditLog.logEvent (per event) | 182,008 |
| AuditLog.anchorBatch (per batch) | 134,747 |

Hardhat EVM (london), optimizer 200 runs; first-use storage costs included.

### *E. Scalability*

Fig. 4 shows goodput and on-chain write load against device count. Central PKI-IAM, ABAC and BC-ZTA served every request up to 200 devices with no failures. The synchronous blockchain kept up to 50 devices (goodput 50 req/s, 1.4 s mean latency) and at 100 devices (100 req/s, 4.2 s), but at 200 devices only 40% of the requests were answered in time, goodput fell to 81 req/s and 4331 requests (summed over 5 runs) failed or timed out. Its on-chain write rate (100 tx/s at 100 and 112 tx/s at 200 devices) is the measured ledger ceiling of this deployment, whereas BC-ZTA writes about 1.0 tx/s regardless of the number of devices. The latency of BC-ZTA stays flat from 10 to 200 devices (5.4 to 4.6 ms mean, P95 at most 6.4 ms), and the PEP used 0.65 of its two vCPUs at 200 devices, so neither the ledger nor the PEP was the limit at this load; larger populations were not tested. For comparison, a sharded design reaches 2,500 TPS at 700 nodes [18]; our approach instead lowers the required write rate.

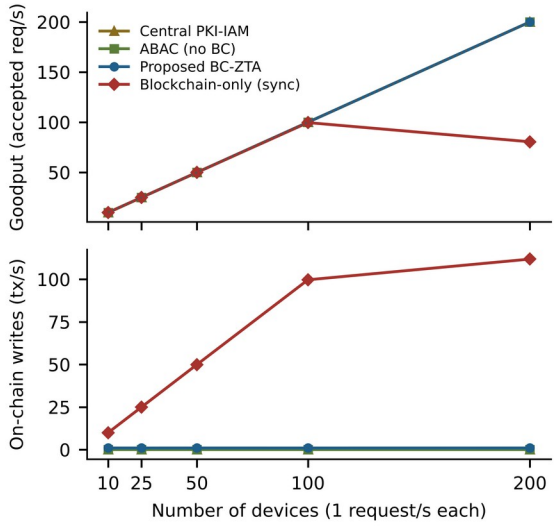



Fig. 4. Goodput (top) and on-chain writes per second (bottom) versus number of devices; dotted line: offered load.

### *F. Attack Resistance*

Table 8 lists the security tests run against the BC-ZTA prototype (5 runs, 50 attempts per test and run). All legitimate requests were accepted (no false rejects), and none of the forged, replayed, stale, unregistered, tampered, class-violating or low-trust requests was accepted. For a revoked device, 0 requests were accepted after the revocation transaction was confirmed, and the first denial came 15 ms after confirmation or sooner. Fig. 5 shows the revocation enforcement delay of each system: with event-driven invalidation BC-ZTA enforces a revocation after 1.25 s on average (worst case 2.23 s), about one block period and independent of the TTL, whereas TTL-only refresh with a 10 s TTL needs 6.6 s on average (worst case 11.5 s). The centralised and local baselines revoke in 57 ms and 22 ms because they hold the state in one place (the ABAC samples are bimodal, about 7 ms or 57 ms; we did not isolate the cause, which lies in the probe path rather than in the store); the blockchain-only value (1.35 s) includes the request that was in flight when the revocation was issued.

TABLE 8. SECURITY TESTS ON THE BC-ZTA PROTOTYPE (5 RUNS)

| Test | Attempts | Accepted |
|---|---|---|
| Legitimate requests (control) | 250 | 250 |
| Forged signature | 250 | 0 |
| Replayed request | 250 | 0 |
| Stale timestamp | 250 | 0 |
| Unregistered DID | 250 | 0 |
| Tampered Merkle proof | 250 | 0 |
| Class outside policy | 250 | 0 |
| Trust score below threshold | 250 | 0 |

For the control row, Accepted is the desired outcome; for all other rows it must be 0.

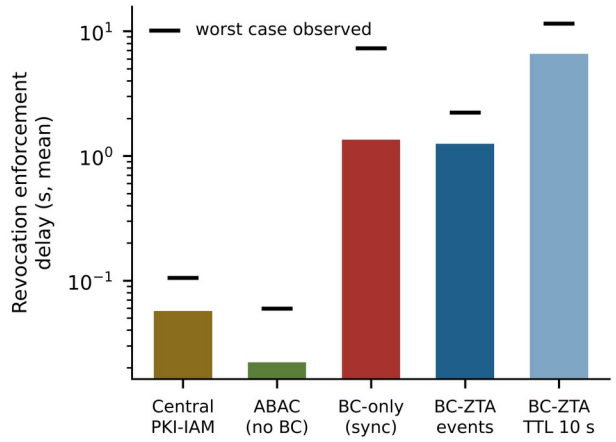



Fig. 5. Revocation enforcement delay (mean, log scale; marks: worst case observed).

### *G. Availability under Validator Failure and Ledger Flooding*

E5 stops one of the four validators for 30 s while 50 devices are served and one device is revoked during the outage. IBFT 2.0 with four validators tolerates $f = 1$ faulty validator and the PEP decides from its cache, so decisions should be unaffected. Table 9 confirms this: all 3500 requests were answered and accepted (mean 5.4 ms, P95 6.8 ms, none late or failed), the revoked device was denied 1.76 s after the revocation was submitted, and the PEP kept anchoring audit batches (73 of 75 anchor transactions were confirmed when the statistics were read). Only one failure of one validator for 30 s was tested; network partitions, Byzantine validators and the loss of two validators were not.

TABLE 9. VALIDATOR-FAILURE TEST (E5, ONE RUN)

| Measure | Value |
|---|---|
| Requests answered / accepted | 3500 / 3500 |
| Mean / P95 latency (ms) | 5.4 / 6.8 |
| Late or failed requests | 0 / 0 |
| Revocation enforced (s) | 1.76 |
| Anchors sent / confirmed | 75 / 73 |

One of four validators stopped for 30 s of a 70 s window; 50 devices at 1 req/s.

In E4 the ledger is flooded with 200 tx/s of audit-log transactions for 35 s while 50 devices are served. Besu accepted 3166 to 7000 of the 7000 flood transactions per run and rejected the rest (presumably a full transaction pool), so the accepted flood rate was 90 to 200 tx/s. In all 5 runs the PEP answered every request (success 100%), with mean latency 4.9–5.1 ms against 5.0–5.4 ms with an idle ledger. This is the intended effect of keeping the ledger off the request path. On the shared CI host the same flood had caused request failures, which we attribute to CPU contention between the PEP and the flooded validators and which the separate hosts removed.

*H. Comparison with Production IAM Services*

To replace the simple directory of B1 by real products, we added Keycloak 26.4 as an OpenID Connect token-introspection service [43] (B4) and OpenLDAP slapd as a directory queried once per request [44] (B5), each on the directory VM behind the same 40 ms netem delay, plus a variant of each with a 10 s decision cache at the PEP ("cached"). All five systems use the same PEP, signatures and policy. Table 10 gives the results (3 repetitions). At 25 and 100 devices the uncached products need 48.3 ms (Keycloak) and 46.8 ms (OpenLDAP) per decision because every request crosses the emulated WAN, against 6.3 ms for BC-ZTA, i.e. 8× and 7× faster. This advantage is mainly the removal of the WAN round trip: a PEP-side cache of the product's answer reaches 9.8–9.5 ms mean latency, and only its P95 (48 ms, one miss per device per TTL) remains WAN-bound. The decisive difference is therefore not the mean latency but what the cache costs: the cached products enforce a revocation only after the cache expires (mean 5.5 s, worst case 9.5 s for the 10 s TTL), whereas BC-ZTA's event-driven invalidation enforces it in 1.2 s in this run and 1.3 s in the five-run test of Section VI-F (worst case 2.2 s) at the same low latency. The uncached products are faster at revocation, 84 ms (Keycloak) and 72 ms (OpenLDAP), because the central service is consulted on every request; this is the price BC-ZTA pays for edge autonomy, and a plant that needs sub-second revocation must use the uncached products or a faster block period.

TABLE 10. PRODUCTION IAM BASELINES: LATENCY (MEAN / P95, MS) AND REVOCATION DELAY

| System | 25 dev. | 100 dev. | 200 dev. | Revoc. (s) |
|---|---|---|---|---|
| Keycloak | 49.7 / 52 | 48.3 / 50 | 123.4 / 304 | 0.08 |
| Keycloak, cached | 10.2 / 49 | 9.8 / 48 | 12.4 / 56 | 5.55 |
| OpenLDAP | 47.3 / 49 | 46.8 / 49 | 77.2 / 243 | 0.07 |
| OpenLDAP, cached | 10.1 / 47 | 9.5 / 47 | 10.6 / 50 | 5.20 |
| BC-ZTA | **6.4 / 7** | **6.3 / 8** | 117.2 / 255 | **1.18** |

Mean of 3 repetitions; 40 ms netem delay on the IAM services; revocation: 8 devices per run, mean of 24; cache TTL 10 s.

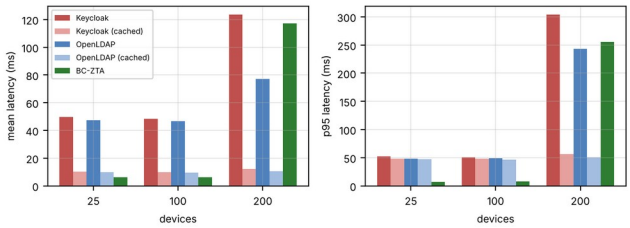



Fig. 6. Mean (left) and P95 (right) latency of Keycloak, OpenLDAP, their cached variants and BC-ZTA.

At 200 devices (one request per second each) the PEP, a single Node.js process on a 2-vCPU VM, approaches its own capacity: BC-ZTA (117 ms mean, 255 ms P95) and uncached Keycloak (123 ms) degrade similarly, while the cached products do not. In the earlier run of Section VI-A the same BC-ZTA configuration still decided in 4.6 ms at 200 devices, so this point lies at the saturation edge and is sensitive to run-to-run variation; we report it as a limit of our single-process PEP, not of the architecture, since PEPs are stateless and can be replicated per gateway. An open-loop capacity test (200–1,600 req/s, 3 repetitions) shows goodput saturating at about 246–291 req/s for all three systems, with P95 latency of several seconds beyond saturation; per-PEP capacity is therefore bounded by request signature verification and not by the choice of IAM backend.

Table 11 reports the outage test: the central service is stopped 20 s into a 100 s run with 100 devices and restarted at 45 s. Without Keycloak or OpenLDAP no new access decisions can be made: 0% of requests were granted during the outage for both uncached products, and 4% for the cached variants, whose entries expired within 2 s of the stop (they kept serving only until the 10 s TTL ran out and, because requests are spread over the TTL, most entries had already expired). Service resumed 1 s after restarting OpenLDAP and 14 s after restarting Keycloak (JVM start-up). BC-ZTA, whose PEP does not contact any central service, granted 100% throughout (Fig. 7). This test removes a service that BC-ZTA does not use; a design that caches decisions and receives pushed revocations from the same directory would also survive a short outage, so the fair comparison is the loss of each design's own state source (Section VI-J). We did not test replicated or clustered Keycloak/OpenLDAP deployments, which would reduce this effect at the cost of operating them; the test shows the single-point-of-failure property that the architecture removes, not that central IAM cannot be made highly available.

TABLE 11. CENTRAL-SERVICE OUTAGE (25 s, 100 DEVICES, 3 RUNS)

| System | Granted during outage (%) | Recovery after restart (s) |
|---|---|---|
| Keycloak | 0 | 14 |
| Keycloak, cached | 4 | 14 |
| OpenLDAP | 0 | 1 |
| OpenLDAP, cached | 4 | 1 |
| BC-ZTA | **100** | **0** |

Service stopped at 20 s and restarted at 45 s of a 100 s run; BC-ZTA has no central service (Keycloak was stopped on the directory VM, but BC-ZTA does not use it).

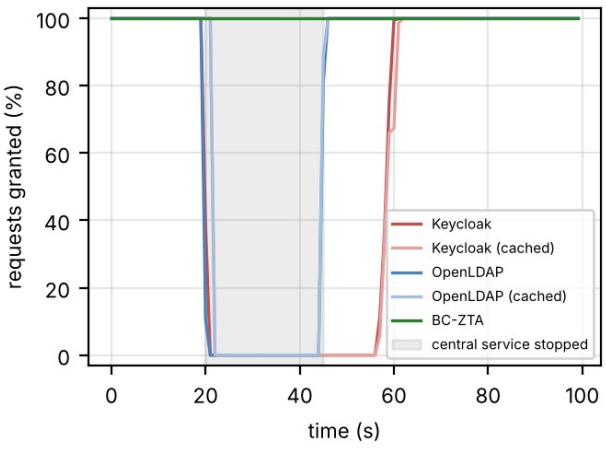



Fig. 7. Fraction of requests granted over time while the central IAM service is stopped (shaded) and restarted.

*I. Ledger Flood Sweep*

Section VI-G tested one flood rate. We therefore swept the offered rate of audit-log transactions from 0 to 1,200 tx/s (16 spam wallets, pre-signed transactions sent in nonce order, 20 s per rate after a 5 s warm-up, 50 devices at 1 request/s, 2 repetitions) once with BC-ZTA and once with the synchronous design, which writes each decision as a transaction on the same ledger (Table 12, Fig. 8). The committed rate saturates at about 193 tx/s for BC-ZTA and 215 tx/s for the synchronous design (peak at 200–400 tx/s offered) and then declines, to 106 and 152 tx/s at 1,200 offered, as Besu rejects most of the excess ("nonce too distant" and, at the highest rate, "transaction pool not enabled"; 50266 of 64802 submissions in the BC-ZTA run). BC-ZTA's access latency does not follow: its mean stays at 5.8–6.3 ms and its P95 at no more than 7.9 ms with every request answered, because decisions never wait for the ledger. The synchronous design's mean latency rises from 1.6 s with an idle ledger to 24 s at 1,200 tx/s (P95 30 s), and 7% of its requests fail at the highest rate. The flood therefore denies service to ledger-bound designs but only delays BC-ZTA's own audit anchoring and revocation visibility, which we did not measure under flood.

TABLE 12. LEDGER FLOOD SWEEP (2 RUNS PER RATE)

| Offered (tx/s) | Commit (tx/s) | BC-ZTA P95 (ms) | Sync. mean (s) | Sync. P95 (s) |
|---|---|---|---|---|
| 0 | 1 | 6.9 | 1.6 | 2.5 |
| 100 | 97 | 6.6 | 2.9 | 4.5 |
| 200 | 184 | 6.5 | 10.2 | 14.7 |
| 400 | 193 | 6.8 | 18.4 | 24.6 |
| 800 | 162 | 6.9 | 24.2 | 31.4 |
| 1200 | 106 | 7.9 | 24.4 | 30.0 |

Commit = committed tx/s of the BC-ZTA run, from block data. Sync. = synchronous on-chain design.

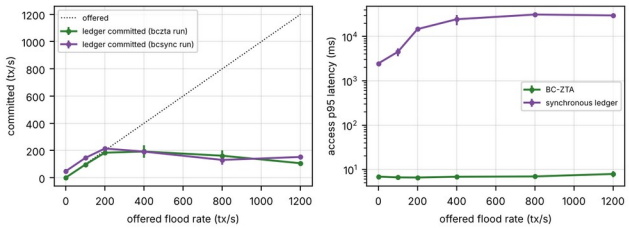



Fig. 8. Committed ledger rate (left) and access P95 latency (right) versus offered flood rate.

The flood generator targets the validator that the PEP also reads from, and the Besu heap was raised to 2.5 GB after a first attempt with the default 1 GB heap ran out of memory in the synchronous run at 400 tx/s; a flood against all validators, an attacker with more accounts, or a longer flood were not tested. The committed rate of 100–215 tx/s is specific to our four-validator, 2 s block, two-vCPU validator setup.

*J. Robustness: Push Revocation, Read Quorum, Fail-Closed Bound, Consensus Halt and Scaling*

The comparison with Keycloak and OpenLDAP leaves open whether the ledger is needed for what BC-ZTA does well. We therefore added baseline B6, a central directory (40 ms WAN delay) whose answers the PEP caches for 30 s and which pushes each revocation to the PEP over a server-sent-event stream, i.e. event-driven invalidation without a blockchain, and BC-ZTA with a read quorum of $f + 1 = 2$ validators (Section III-K). The experiments were repeated 3 times on a fresh network each; Table 13 gives the results. B6 decides in 3.9 ± 0.4 ms at 100 devices against 4.2 ± 0.4 ms for BC-ZTA, so the two are equivalent for access latency. B6 enforces a revocation after 109 ms on average (worst case 154 ms), which is 11× faster than BC-ZTA with quorum (1.2 s, worst case 2.1 s), because BC-ZTA must wait for a block. The read quorum changes the mean latency at 100 devices from 4.2 ± 0.4 to 4.6 ± 1.1 ms, since only cache misses read from two nodes, and no quorum mismatch occurred in the runs. The result is that a trusted central operator who pushes revocations matches BC-ZTA's latency and revokes faster; what the ledger provides that B6 cannot is that no single operator holds identity, policy and audit records, and that anchored audit can be verified by the other consortium members. Where one operator is acceptable, B6 is the simpler design.

TABLE 13. CENTRAL DIRECTORY WITH PUSH REVOCATION (B6) AND READ QUORUM (3 RUNS)

| System | 25 dev. | 100 dev. | 200 dev. | Revoc. (ms) |
|---|---|---|---|---|
| B6 push | 4.7 ± 0.3 | 3.9 ± 0.4 | 3.6 ± 0.3 | 109 |
| BC-ZTA | 4.5 ± 0.5 | 4.2 ± 0.4 | 4.0 ± 0.3 | see Table 10 |
| BC-ZTA, quorum 2 | 4.6 ± 0.2 | 4.6 ± 1.1 | 4.0 ± 0.5 | 1162 |

Mean latency (ms) ± 95% CI of 3 runs; revocation: mean of 24 (B6) and 24 (quorum) samples. TTL 30 s (latency), 10 s (revocation).

Section VI-H stopped Keycloak or OpenLDAP, a service BC-ZTA does not use. The fair test is the loss of each design's own state source with the same cache policy (TTL 10 s, serve-stale bound 2·TTL = 20 s, 50 devices): the PEP of BC-ZTA is cut off from its ledger node with a firewall rule (*iptables* DROP) for 50 s, and the directory of B6 is stopped for the same time (Table 14, Fig. 9, 3 runs). Both designs behave alike: requests are granted from cache and then from stale state (89% and 100% granted in the first 10 s after the loss), granted fractions fall to 0% and 0% once the last read is older than 20 s, and service resumes 1 s and 1 s after the source returns. The first denials appear 9 s (BC-ZTA) and 12 s (B6) after the loss rather than after 20 s, because the bound of each cache entry runs from its own last successful read, which lies up to one TTL before the loss; the first reads that hit the unreachable ledger also wait for the 2.5 s read timeout, which is why BC-ZTA grants slightly less than B6 in the first window. The ledger therefore gives no extra availability by itself in this test; any cache with a stale bound provides the same short-outage behaviour. What differs is that the BC-ZTA state is replicated on four validators and the PEP could read from another node, which we did not implement as automatic failover in this test, while the directory of B6 is a single service unless it is replicated too.

TABLE 14. LOSS OF THE STATE SOURCE (TTL 10 S, STALE BOUND 20 S, 3 RUNS)

| System (source lost) | Before | ≤ TTL | TTL–2TTL | > 2TTL | Restored |
|---|---|---|---|---|---|
| BC-ZTA (ledger node) | 100 | 89 | 15 | 0 | 100 |
| B6 (directory) | 100 | 100 | 20 | 0 | 100 |

Requests granted (%) in the windows after the loss of the state source; windows are relative to the time of the loss.

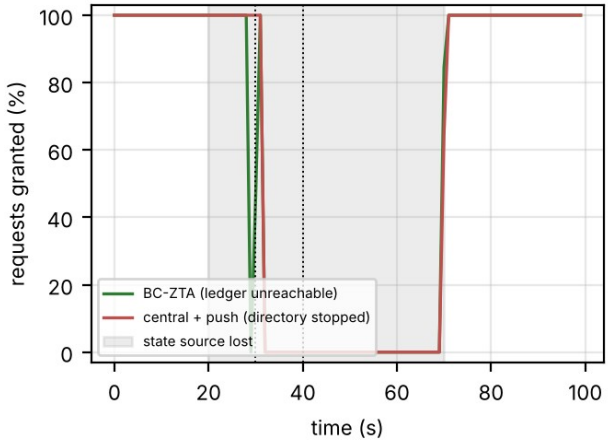



Fig. 9. Requests granted while the state source is lost (shaded); dotted lines: TTL and 2·TTL after the loss.

To test the consensus assumption we stopped two of the four validators for 40 s (IBFT 2.0 needs three, so block production halts) while 50 devices were served and a device was revoked 5 s after the stop. The PEP kept answering: 100% of the requests were granted during the halt, as the cache and the still-reachable node serve the last committed state. The revoked device, however, stayed accepted until consensus resumed and the transaction was mined: enforcement came 126.2 s after the validators were restarted and 161.1 s after submission (3 runs). The staleness bound of BC-ZTA therefore runs from finalisation of the revocation, not from its submission, and a consensus halt is a window in which revocations cannot take effect; a central directory is not affected by this particular failure. The delay is much longer than the halt itself (40 s): block production stayed frozen for 126 s after the validators were restarted. This is consistent with the round-change rule of IBFT 2.0, in which the timeout of a round doubles after each failed round (base 4 s in our configuration), so the surviving and the restarted validators needed one long round to agree again; shorter base timeouts or supervised automatic restarts of validators would reduce it, and an operator that needs bounded revocation latency during a halt must additionally enforce revocations at the PEP by a short-lived local deny list.

Finally, per-PEP capacity is not a limit of the architecture if gateways are replicated. We ran k = 1, 2, 4 PEP processes on one 4-vCPU VM, with devices sharded across them, and raised the offered rate (Fig. 10, Table 15). The highest offered rate served without any failed request and with a P95 below 50 ms was 200 req/s for k = 1, 400 req/s for k = 2, 400 req/s for k = 4. At 800 req/s offered, k = 1 is overloaded (goodput 457 req/s, P95 8.5 s) and k = 2 is overloaded (goodput 706 req/s, P95 7.2 s), whereas k = 4 delivers 770 req/s (96% of the offered rate) at a P95 of 436 ms while using 2.4 cores of the 4-vCPU VM. Beyond that the goodput of every configuration falls as the offered rate rises (overload collapse: requests queue behind signature verification and time out), so a deployment must keep the rate per gateway below its knee, which an admission limit or load balancer would enforce. Scale-out was measured on one host; across hosts the shared state is the ledger, which is read once per device per TTL.

TABLE 15. PEP SCALE-OUT (BC-ZTA, 3 RUNS)

| PEPs | Clean rate (req/s) | Goodput at 800 (req/s) | P95 at 800 (ms) | CPU cores at 800 |
|---|---|---|---|---|
| 1 | 200 | 457 | 8464 | 0.7 |
| 2 | 400 | 706 | 7172 | 1.2 |
| 4 | 400 | 770 | 436 | 2.4 |

Clean rate = highest offered rate with no failed request and P95 < 50 ms; one VM; load generator on a separate VM; mean of the runs.

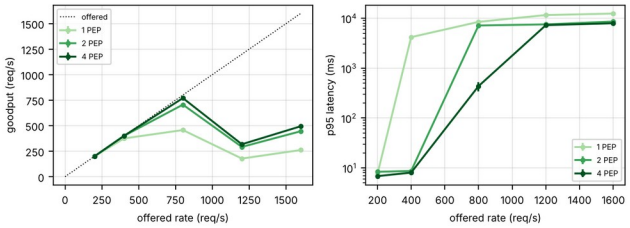



Fig. 10. Goodput (left) and P95 latency (right) versus offered request rate for 1, 2 and 4 PEP processes.

### *K. Comparison with Baselines*

BC-ZTA is 0.2 ms slower than a ledger-free ABAC store at 50 devices but adds decentralised identity, shared policy and tamper-evident anchored audit that B2 lacks; against the centralised PKI-IAM it is 89% faster at 50 devices (with a 40 ms netem delay on the directory server) and removes the central directory; and against the synchronous blockchain it avoids ~1.4 s per decision at 50 devices and the ledger ceiling. Against real Keycloak and OpenLDAP services (Section VI-H) it is 8× and 7× faster at 100 devices, but a PEP-side cache gives similar mean latency at the cost of stale

revocations, and a central directory with pushed revocations (Section VI-J) matches its latency and revokes faster. Its price is revocation latency: about 1.3 s with events (one block) instead of about 84 ms for a central store. A reported Ethereum-based framework pays about 25 ms extra per request even at small scale [17], consistent with our finding that on-path consensus is the bottleneck.

### *L. Discussion and Limitations*

The key trade-off is bounded revocation staleness for latency. The push baseline of Section VI-J shows that this trade-off is not specific to the ledger: a trusted central operator obtains the same latency and a faster revocation, so the ledger has to be justified by who controls the identity state and who can verify the audit, not by speed. Limitations: (i) the seven VMs sit in one zone of one cloud provider, so inter-VM delay is that of a data-centre network plus the injected netem delays, not a plant network or a WAN, and absolute latencies and throughput are specific to these VM sizes; (ii) B1–B3 are simple re-implementations; Keycloak ran in development mode (single node, embedded H2 database) and OpenLDAP as a single server, both behind an emulated 40 ms WAN, so production-tuned or clustered deployments may behave differently, particularly for availability; (iii) the device population is emulated by a load generator with up to 200 devices at 1 request/s: no physical devices, industrial protocols, secure elements, larger populations or bursty and churn-heavy workloads were used; (iv) one plant authority key issues registrations and revocations, so authority compromise is out of scope; (v) the availability tests are limited: the E4 flood was partly rejected by Besu (an accepted rate of 90–200 tx/s) and E5 stops one validator once for 30 s and Section VI-J stops two for 40 s, with no network partition or Byzantine validator; (vi) each experiment has five repetitions (three for the IAM tests and two for the flood sweep) on the same VMs, the revocation tests use 8 devices per run, and a single-process PEP saturates at roughly 250–290 req/s, so larger sites need several PEPs (scale-out was measured on one host only); and (vii) trust scores are voted values, not yet learned from data. Comparisons with published numbers are indicative only, since workloads and testbeds differ.

Further limitations follow from the security analysis (Section IV-F). The nonce cache is per gateway and the signed request carries no gateway identifier, so a captured request can be replayed once at another gateway within $\Delta$; the default read path trusts one ledger node (the quorum option is implemented, and its cost is measured in Section VI-J, but it is not part of the default measurements); audit completeness is not protected against a compromised gateway; the Merkle epoch proof shows enrolment, not freshness; and the device-side cost of signing and proof storage was not measured, so the claim of compatibility with constrained devices (D5) is an argument, not a result. Each request is signed individually, which is appropriate for access decisions but not for high-rate telemetry, where a session credential issued after one verified request would be used. All statistical statements rest on 2–5 repetitions with standard deviations or 95% confidence intervals and no hypothesis tests.

Deployment considerations. First, validators must be run by independent parties (plant, integrator, vendor) for the Byzantine assumption to mean anything. Second, device keys should live in a secure element or TPM, because the architecture's identity guarantees are only as strong as key custody. Third, on-chain records contain key hashes, scores and event hashes, not process data, which keeps operational data off the shared ledger. Fourth, in the prototype AuditLog and grantSession are callable by any account, and production deployments would restrict them to gateway identities. Fifth, the fail-closed rule of Section III-J trades availability for safety during ledger partitions and should be tuned against plant uptime requirements.

## VII. CONCLUSION

We presented BC-ZTA, a blockchain-assisted zero-trust architecture that keeps consensus off the per-request path through edge-cached, Merkle-verified state and asynchronous batched anchoring. Four Solidity contracts pass 15 unit tests, and measured gas shows batch anchoring is 68–1351× cheaper per record than per-event logging for batches of 50–1,000 records. In end-to-end tests on a seven-VM Google Cloud deployment with a four-validator Besu network, BC-ZTA decided in 5.4 ms on average at 50 devices and served 200 devices without failures with about one on-chain write per second, and kept deciding while one validator was stopped, while the synchronous design needed 1.4 s at 50 devices and answered only 40% of the requests at 200 devices; the price is a revocation delay of about 1.3 s with event-driven invalidation. A baseline with a central directory and pushed revocations, which we added to test whether the ledger is needed for speed, has the same latency and revokes in 109 ms; the case for the ledger therefore rests on removing any single operator's control over identity state and on audit that third parties can verify, not on performance. A fail-closed bound of twice the cache lifetime, a read quorum of two validators, replicated gateways and the behaviour during a consensus halt (revocations wait for consensus, here 161 s on average) are evaluated in Section VI-J. All forged, replayed, stale, unregistered, tampered and policy-violating requests were rejected. The evaluation used cloud VMs in a single zone with injected network delay and an emulated device population. Future work includes a multi-site Besu deployment with real network delay, physical devices and industrial protocols, clustered production IAM baselines, learning-based trust scoring and post-quantum signatures.

## ACKNOWLEDGMENT

[Funding sources and acknowledgments to be added by the authors.]

## DATA AND CODE AVAILABILITY

The Solidity contracts, the PEP and load-generator code, the Google Cloud deployment scripts and all raw results used in this paper are available at https://github.com/irfanrazanaqvi/bc-zta-iiot-experiments (archived release with DOI: [to be added by the authors before submission]).

## REFERENCES


[1] F. Federici, D. Martintoni, and V. Senni, “A zero-trust architecture for remote access in industrial IoT infrastructures,” Electronics, vol. 12, no. 3, Art. no. 566, 2023, doi: 10.3390/electronics12030566.

[2] C. Zanasi, S. Russo, and M. Colajanni, “Flexible zero trust architecture for the cybersecurity of industrial IoT infrastructures,” Ad Hoc Netw., vol. 156, Art. no. 103414, 2024, doi: 10.1016/j.adhoc.2024.103414.

[3] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero trust architecture,” Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Spec. Publ. 800-207, Aug. 2020, doi: 10.6028/NIST.SP.800-207.

[4] M. L. Gambo and A. Almulhem, “Zero trust architecture: A systematic literature review,” J. Netw. Syst. Manage., vol. 34, Art. no. 25, 2026, doi: 10.1007/s10922-025-09998-x.

[5] S. Mushtaq, M. Mohsin, and M. M. Mushtaq, “A systematic literature review on the implementation and challenges of zero trust architecture across domains,” Sensors, vol. 25, no. 19, Art. no. 6118, 2025, doi: 10.3390/s25196118.

[6] M. A. Azad, S. Abdullah, J. Arshad, H. Lallie, and Y. H. Ahmed, “Verify and trust: A multidimensional survey of zero-trust security in the age of IoT,” Internet Things, vol. 27, Art. no. 101227, 2024, doi: 10.1016/j.iot.2024.101227.

[7] S. Mthethwa, M. T. Dlamini, and E. Jembere, “Zero trust for NHIs based on robust identity and access management for a resilient IoT future,” Sensors, vol. 26, no. 8, Art. no. 2392, 2026, doi: 10.3390/s26082392.

[8] M. Asim, N. Tariq, A. I. Awad, F. Waheed, S. Bukhari, A. Rafay, and H. H. Song, “Lightweight and privacy-aware decentralized identity management for Industrial Internet of Things applications,” Future Gener. Comput. Syst., vol. 186, Art. no. 108753, 2027 (online Aug. 2026), doi: 10.1016/j.future.2026.108753.

[9] E. B. Fernández and A. Brazhuk, “A critical analysis of zero trust architecture (ZTA),” Comput. Stand. Interfaces, vol. 89, Art. no. 103832, 2024, doi: 10.1016/j.csi.2024.103832.

[10] Y.-W. Ma and P.-H. Chiu, “A novel risk-based access control engine in zero trust architecture for IoT network,” Int. J. Inf. Secur., vol. 24, Art. no. 124, 2025, doi: 10.1007/s10207-025-01030-2.

[11] M. A. Aleisa, “Blockchain-enabled zero trust architecture for privacy-preserving cybersecurity in IoT environments,” IEEE Access, vol. 13, pp. 18660–18676, 2025, doi: 10.1109/ACCESS.2025.3529309.

[12] S. Nie, J. Ren, R. Wu, P. Han, Z. Han, and W. Wan, “Zero-trust access control mechanism based on blockchain and inner-product encryption in the Internet of Things in a 6G environment,” Sensors, vol. 25, no. 2, Art. no. 550, 2025, doi: 10.3390/s25020550.

[13] G. Ali, S. Shah, M. Elaffendi, and N. Ahmad, “Blockchain-based access management framework for interoperable digital twins in industrial IoT,” Front. Blockchain, vol. 8, Art. no. 1693926, 2025, doi: 10.3389/fbloc.2025.1693926.

[14] M. Essaid and H. Ju, “Blockchain solutions for enhancing security and privacy in Industrial IoT,” Appl. Sci., vol. 15, no. 12, Art. no. 6835, 2025, doi: 10.3390/app15126835.

[15] D. Li, H. Li, N. Crespi, R. Minerva, M. Li, W. Liang, and K. C. Li, “Hyper-IIoT: A smart contract-inspired access control scheme for resource-constrained Industrial Internet of Things,” IEEE Trans. Sustain. Comput., vol. 10, no. 5, pp. 820–829, 2025, doi: 10.1109/TSUSC.2025.3542466.

[16] Z. Guo, “Blockchain-enhanced smart contracts for formal verification of IoT access control mechanisms,” Alexandria Eng. J., vol. 118, pp. 315–324, 2025, doi: 10.1016/j.aej.2024.12.109.

[17] A. Singh, V. Pareek, and A. Sharma, “Blockchain-enabled zero trust framework for securing FinTech ecosystems against insider threats and cyber attacks,” arXiv:2507.19976, 2025.

[18] M. M. Salim, M. Kim, S. K. Singh, and J. H. Park, “Zero-trust blockchain-enabled framework for scalable and secure IoT networks,” Future Gener. Comput. Syst., vol. 175, Art. no. 108093, 2026, doi: 10.1016/j.future.2025.108093.

[19] A. Sharma, S. Rani, and W. Boulila, “Blockchain-based zero trust networks with federated transfer learning for IoT security in industry 5.0,” PLOS ONE, vol. 20, no. 6, Art. no. e0323241, 2025, doi: 10.1371/journal.pone.0323241.

[20] S. Mishra, T. S. M. Aldafas, and N. S. Alshammari, “A zero-trust digital twin framework for privacy-preserving multi-dataset intrusion detection in industrial IoT with lightweight blockchain auditing,” Sci. Rep., vol. 16, Art. no. 15236, 2026, doi: 10.1038/s41598-026-42041-w.

[21] Z. Zhai, J. Liu, X. Liu, Y. Mao, X. Zhang, J. Ma, and C. Jin, “A lightweight authentication method for Industrial Internet of Things based on blockchain and Chebyshev chaotic maps,” Future Internet, vol. 17, no. 8, Art. no. 338, 2025, doi: 10.3390/fi17080338.

[22] T. Ramírez-Gordillo, A. Maciá-Lillo, F. A. Pujol, N. García-D’Urso, J. Azorín-López, and H. Mora, “Decentralized identity management for Internet of Things (IoT) devices using IOTA blockchain technology,” Future Internet, vol. 17, no. 1, Art. no. 49, 2025, doi: 10.3390/fi17010049.

[23] S. I. R. Naqvi, Z. J. Bin, A. Mohsin, M. P. Akhter, and M. M. ur Rahman, “DBoT: Enhancing scalability and privacy in intelligent building spaces through a direct acyclic graph blockchain,” in Proc. 17th Int. Conf. Open Source Syst. Technol. (ICOSST), 2023, doi: 10.1109/ICOSST60641.2023.10414234.

[24] R. Islam, R. Bose, S. Roy, A. A. Khan, S. Sutradhar, S. Das, F. Ali, and A. A. AlZubi, “Decentralized trust framework for smart cities: A blockchain-enabled cybersecurity and data integrity model,” Sci. Rep., vol. 15, Art. no. 23454, 2025, doi: 10.1038/s41598-025-06405-y.

[25] P. Rani, “A zero-trust framework for securing next-generation Internet of Things network,” in New Trends and Applications in Internet of Things (IoT) and Big Data Analytics, vol. 2. Cham, Switzerland: Springer, 2026, pp. 213–225, doi: 10.1007/978-3-032-21753-0_12.

[26] B. Chellaprabha, D. Manohari, and J. J. Justus, “Scalable and secure IoT architecture using trust-driven adaptive resource management and edge intelligence,” in Expert Applications and Security (NICE-TEAS 2025), Lecture Notes in Networks and Systems, vol. 1823. Cham, Switzerland: Springer, 2026, pp. 444–461, doi: 10.1007/978-3-032-20114-0_40.

[27] K. Li et al., “Zero-trust foundation models: A new paradigm for secure and collaborative artificial intelligence for Internet of Things,” arXiv:2505.23792, 2025.

[28] S. Ghasemshirazi, G. Shirvani, and M. A. Alipour, “Zero trust: Applications, challenges, and opportunities,” arXiv:2309.03582, 2023.

[29] A. K. Peepliwal et al., “A prototype model of zero-trust architecture blockchain with EigenTrust-based practical Byzantine fault tolerance protocol to manage decentralized clinical trials,” arXiv:2408.16885, 2024.

[30] A. Kulkarni, N. A. Hazari, and M. Y. Niamat, “A zero trust-based framework employing blockchain technology and ring oscillator physical unclonable functions for security of field programmable gate array supply chain,” IEEE Access, vol. 12, pp. 89322–89338, 2024. [Online]. Available: https://scholars.utoledo.edu/display/pub-001262650600001

[31] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk, “Internet X.509 public key infrastructure certificate and certificate revocation list (CRL) profile,” Internet Eng. Task Force, RFC 5280, May 2008. [Online]. Available: https://www.rfc-editor.org/rfc/rfc5280

[32] S. Santesson, M. Myers, R. Ankney, A. Malpani, S. Galperin, and C. Adams, “X.509 Internet public key infrastructure online certificate status protocol—OCSP,” Internet Eng. Task Force, RFC 6960, Jun. 2013. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6960

[33] D. Eastlake 3rd, “Transport layer security (TLS) extensions: Extension definitions,” Internet Eng. Task Force, RFC 6066, Jan. 2011. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6066

[34] B. Laurie, A. Langley, and E. Kasper, “Certificate transparency,” Internet Eng. Task Force, RFC 6962, Jun. 2013. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6962

[35] M. Sporny, D. Longley, M. Sabadello, D. Reed, O. Steele, and C. Allen, “Decentralized identifiers (DIDs) v1.0,” W3C Recommendation, Jul. 2022. [Online]. Available: https://www.w3.org/TR/did-core/ [Online]. Available: https://www.w3.org/TR/did-core/

[36] A. Ouaddah, A. Abou Elkalam, and A. Ait Ouahman, “FairAccess: A new blockchain-based access control framework for the Internet of Things,” Secur. Commun. Netw., vol. 9, no. 18, pp. 5943–5964, 2016, doi: 10.1002/sec.1748.

[37] O. Novo, "Blockchain meets IoT: An architecture for scalable access management in IoT," IEEE Internet Things J., vol. 5, no. 2, Apr. 2018, doi: 10.1109/JIOT.2018.2812239.

[38] E. Androulaki et al., "Hyperledger Fabric: A distributed operating system for permissioned blockchains," in Proc. 13th EuroSys Conf., Porto, Portugal, Apr. 2018, Art. no. 30, doi: 10.1145/3190508.3190538.

[39] H. Moniz, "The Istanbul BFT consensus algorithm," arXiv:2002.03613, 2020.

[40] M. Castro and B. Liskov, "Practical Byzantine fault tolerance," in Proc. 3rd USENIX Symp. Oper. Syst. Design Implement. (OSDI), New Orleans, LA, USA, Feb. 1999, pp. 173–186. [Online]. Available: https://www.usenix.org/conference/osdi-99/practical-byzantine-fault-tolerance

[41] M. Dahlmanns, J. Lohmöller, I. B. Fink, J. Pennekamp, K. Wehrle, and M. Henze, "Easing the conscience with OPC UA: An Internet-wide study on insecure deployments," in Proc. ACM Internet Meas. Conf. (IMC), Virtual Event, Oct. 2020, pp. 101–110, doi: 10.1145/3419394.3423666.

[42] B. Blanchet, "Modeling and verifying security protocols with the applied pi calculus and ProVerif," Found. Trends Privacy Secur., vol. 1, no. 1–2, pp. 1–135, 2016, doi: 10.1561/3300000004.

[43] J. Richer, Ed., "OAuth 2.0 token introspection," Internet Eng. Task Force, RFC 7662, Oct. 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7662

[44] J. Sermersheim, Ed., "Lightweight Directory Access Protocol (LDAP): The protocol," Internet Eng. Task Force, RFC 4511, Jun. 2006. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4511

## AUTHOR BIOGRAPHIES

**[AUTHOR NAME]** [Short biography, degrees, current position and research interests to be added; IEEE Access also requires a photo.]